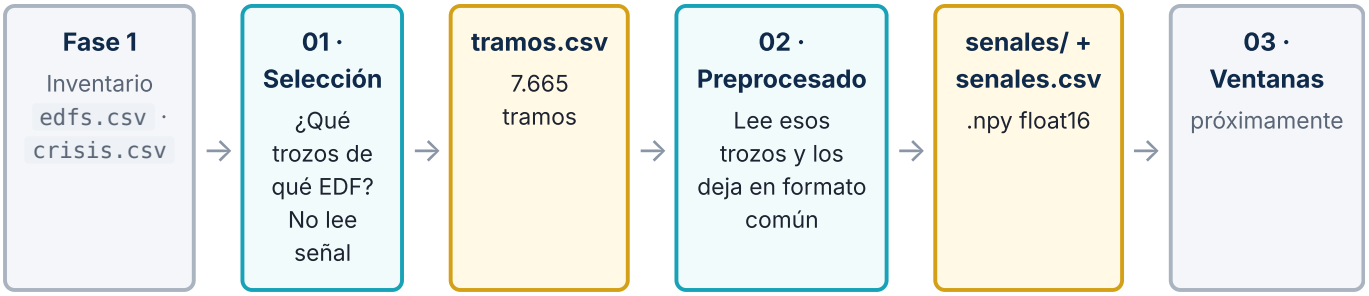
TFM · Predicción y clasificación de crisis epilépticas con EEG (TUSZ v2.0.6)

Fase 2 · Cómo funcionan los notebooks 01 y 02

Guía didáctica de `01_seleccion_crisis` y `02_preprocesado`



Contenido

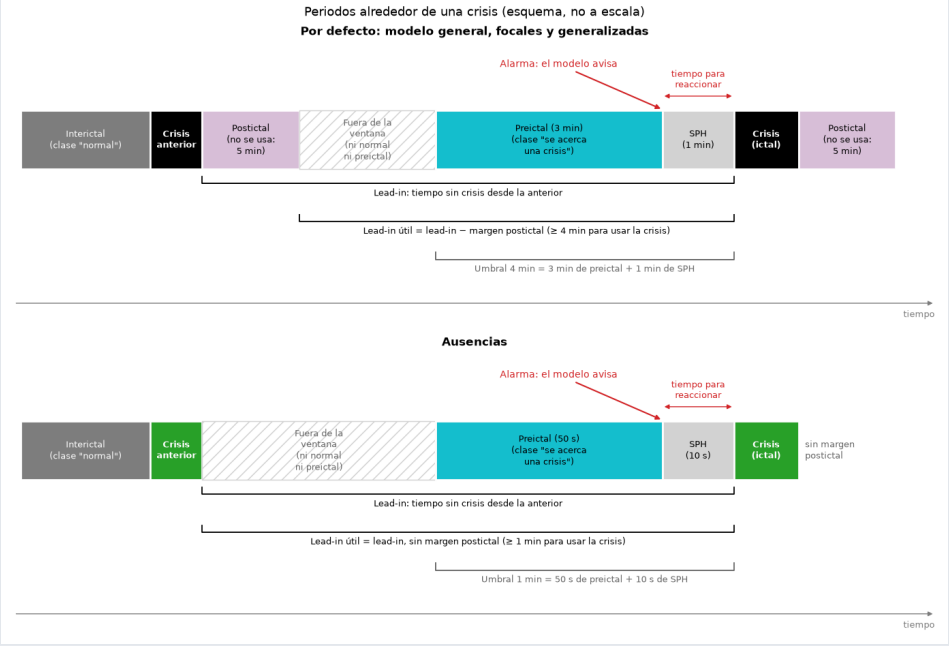
1. Los conceptos que hay que tener claros
2. Notebook 01 · Selección de crisis y tramos
3. Notebook 02 · Preprocesado de la señal
4. Resumen para la defensa: decisiones y por qué

Idea en una línea: **01 decide *qué* señal se usa**; **02 la deja *igual para todos*** (y para la futura demo).

1 · Conceptos clave

Alrededor de cada crisis el tiempo se divide en periodos. Los dos notebooks giran en torno a ellos:

Periodo	Qué es	Uso
Interictal	Señal "normal", lejos de cualquier crisis.	Clase "no se acerca nada" (Fase 3)
Preictal (SOP)	Tramo justo antes de la crisis (sin contar el SPH).	Clase "se acerca una crisis" (Fase 3)
SPH	Último tramo antes de la crisis. **No se usa**: es el tiempo para reaccionar y evita predecir con actividad que ya es de la crisis.	Antelación mínima del aviso
Ictal	La crisis en sí.	Clasificación de tipos (Fase 4)
Margen postictal	Tras una crisis el EEG sigue alterado. **No se usa** para nada.	Depende del tipo de la crisis que acaba
Lead-in	Tiempo sin crisis antes de una crisis, dentro del mismo EDF.	Lo calculó el inventario
Lead-in útil	Lead-in − margen postictal de la crisis anterior.	Decide si la crisis sirve para predecir



Esquema de periodos (figura del notebook 01). Arriba los valores por defecto; abajo las ausencias, con valores propios.

> **La regla que lo decide todo**
> Una crisis se usa en predicción si **lead-in útil ≥ umbral**, donde **umbral = preictal + SPH**.

Los valores (en `tfm_comun/prediccion/umbrales.py`: un solo sitio para cambiarlos)

	Umbral	SPH	Preictal	Margen postictal
Por defecto (modelo general, focales, generalizadas)	4 min	1 min	3 min	5 min
🟩 **absz (🔹) ausencias** (cortas, seguidas, sin postictal)	60 s	10 s	50 s	0 s

2 · Notebook 01 · Selección de crisis y tramos

> **Qué hace, en una frase**
>
> Recorre las crisis del inventario y apunta, **sin leer ningún EDF**, qué trozos (fichero + segundo de inicio + segundo de fin) serán preictal, interictal e ictal. El resultado es `tramos.csv`. Tarda segundos.

Entrada y salida

Entra

- `edfs.csv`: 8.140 ficheros (duración, montaje, fs, partición).
- `crisis.csv`: 2.664 crisis (fichero, inicio, fin, tipo, lead-in).

Sale (en `outputs/fase_02/01_seleccion/`)

- `tramos.csv` ← **el importante**
- recuentos por umbral/tipo/grupo, análisis del umbral, horas por clase, `summary.txt`, figuras.

El recorrido, sección a sección

1. **Librerías y versiones.** Solo pandas, numpy y matplotlib (no hay señal). Importa las utilidades de `tfm_comun` y muestra versiones.
2. **Configuración.** Carga el inventario y monta una tabla con los parámetros de cada tipo (umbral, SPH, margen, grupo). Todo se guarda en `configuracion.json` para saber con qué se ejecutó.
3. **Limpieza.** Descarta 2 crisis, *mostrando cuáles y por qué*: una `sin_tipo` de 0 s y una 🟥 fnsz (♦) de 1,98 s (menos de 2 s no llena ni una ventana). Quedan **2.662**. Calcula el **lead-in útil** y da a cada crisis un `crisis_id`.
4. **Recuento por umbral.** Cuántas crisis y pacientes sobreviven con cada umbral (2, 3, 4, 5, 7, 10, 15 min), por tipo, grupo y partición. Y **elige el umbral** (ver página siguiente).
5. **Tramos preictales** → 695.
6. **Tramos interictales.**
7. **Tramos ictales** → 2.662 (todas las crisis válidas).
8. **Une todo en `tramos.csv`** y pasa **5 comprobaciones** automáticas.
9. **Cuánta señal hay de cada clase** → el desequilibrio.
10–13. **Fichero de ejemplo, resumen, conclusiones y archivos generados.**

El cálculo clave: lead-in útil (sección 3)

```
# Fin de la crisis anterior del mismo fichero (0 si es la primera)
df_crisis['fin_anterior_s'] = df_crisis.groupby('file').stop.shift().fillna(0.0)
# Margen postictal según el TIPO de la crisis anterior
tipo_anterior = df_crisis.groupby('file').label.shift()
df_crisis['margen_postictal_anterior_s'] = [0.0 if pd.isna(t) else margen_postictal_s(t) for t in
tipo_anterior]
# Desde dónde la señal vuelve a ser "limpia" y cuánto hay hasta la crisis
df_crisis['inicio_util_s'] = df_crisis.fin_anterior_s + df_crisis.margen_postictal_anterior_s
df_crisis['leadin_util_s'] = df_crisis.start - df_crisis.inicio_util_s
```

Por qué descontar el margen postictal

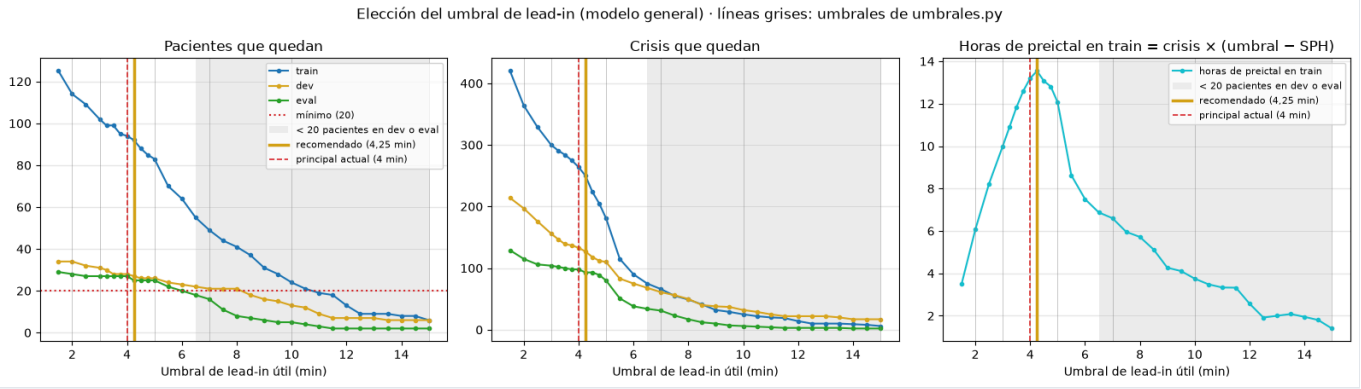
Sin él, el "preictal" podría empezar con EEG todavía alterado por la crisis anterior, y el modelo aprendería "acaba de haber una crisis" en vez de "se acerca una". Cuesta crisis (con 4 min, en train se pasa de 332 a 264), pero limpia la clase. Las ausencias no pierden ninguna (margen 0).

Ojo

Las crisis descartadas **sí cuentan** para los márgenes: aunque no se usen, la señal de alrededor no es normal.

Cómo se eligió el umbral de 4 min (sección 4.3)

Subir el umbral tiene dos efectos opuestos: **cada crisis aporta más preictal** (umbral − SPH), pero **quedan menos crisis y pacientes**. El notebook prueba una rejilla de umbrales y busca el que da **más horas de preictal en train** manteniendo **≥ 20 pacientes en dev y en eval** (para que las métricas no dependan de 4 personas).



Pacientes, crisis y horas de preictal en train según el umbral. Zona gris: menos de 20 pacientes en dev o eval.

Umbral	🔵 train crisis / pac.	🟡 dev pac.	🟢 eval pac.	h preictal train	≥ 20 pac.
2 min	364 / 114	34	28	6,1	✅
3 min	300 / 102	31	27	10,0	✅
4 min ⭐	**264 / 94**	**28**	**27**	**13,2**	✅
5 min	181 / 83	26	25	12,1	✅
7 min	66 / 49	21	16	6,6	❌
10 min	25 / 24	13	5	3,8	❌
15 min	6 / 6	6	2	1,4	❌

Decisión

4 min: casi la misma señal que el máximo exacto (4,25 min) y conserva más pacientes en eval (27 frente a 25). Con 10 y 15 min eval se queda en 5 y 2 pacientes: sirven solo como comparación orientativa.

¿Un modelo por tipo? No. ¿Por grupo? Sí, como experimento (sección 4.2)

Por tipo, solo 🟥 fnsz (♦) y 🟦 gnsz (🔷) tienen pacientes en las tres particiones. Por **grupo de inicio** (cada crisis con los parámetros de su tipo):

Grupo	🔵 train crisis / pac.	🟡 dev	🟢 eval	pac. train+dev
♦️ focal	189 / 70	90 / 18	70 / 16	88
🔷 generalizada	66 / 28	40 / 15	27 / 12	43
🔹 ausencia	30 / 8	22 / 4	2 / 1	12

Ausencias: eval tiene 1 paciente, así que se evaluarán con **validación cruzada por paciente en train+dev (12 pacientes)**.

Los tres tipos de tramo (secciones 5, 6 y 7)

Tramo = un trozo continuo de un EDF con su inicio y fin en segundos. Aquí no se cortan ventanas: solo se apunta qué leer.

Preictal: el tramo más largo posible

Para cada crisis con lead-in útil ≥ 2 min (o ≥ el umbral de su tipo, 60 s en ausencias) se guarda el tramo que va **desde el fin del margen postictal anterior hasta el inicio del SPH**.

> **Ejemplo**
> 🟥 fnsz (♦️): la crisis anterior acaba en el min 2 y esta empieza en el 12. Postictal hasta el 7 → lead-in útil 5 min (≥ 4, entra). Se guarda del **7 al 11**; el modelo general usará los 3 últimos min (8–11); el SPH va del 11 al 12.

> **Por qué el tramo más largo y no la ventana final**
> Así **una sola tabla vale para todos los umbrales y modelos**. Columnas como `cumple_4min` o `cumple_umbral_tipo` dicen a `03_ventanas` qué recortar para cada modelo.

Test común: las 115 crisis de 🟢 eval (28 pacientes) con ≥ 2 min se evalúan, para todos los umbrales, en el mismo minuto (de 2 a 1 min antes). Si no, un umbral podría ganar solo por tener un test más fácil.

Interictal: lejos de todo

Se prohíbe, alrededor de cada crisis, **15 min antes** (el umbral más alto) y el **margen postictal después**. Lo que queda es interictal. Sale de ficheros sin crisis (casi todo: 371 h en train) y de huecos lejos de crisis en ficheros con crisis.

```
# Trozos de [inicio, fin] que no tocan ningún intervalo prohibido
def restar_intervalos(inicio, fin, prohibidos):
    libres, cursor = [], inicio
    for a, b in sorted(prohibidos):
        if a > cursor:
            libres.append((cursor, min(a, fin)))
        cursor = max(cursor, b)
        ...
# Prohibido: 15 min antes de cada crisis y su margen postictal después
prohibidos = [(a - MARGEN_PREVIO_S, b + margen_postictal_s(t)) for a, b, t in zip(cr.start, cr.stop,
cr.label)]
```

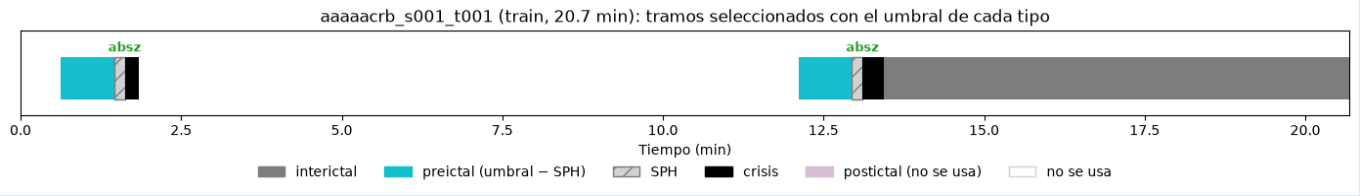
> **Ejemplo**
> Fichero de 30 min con una crisis del min 20 al 21. Se quita 5–20 (antes) y 21–26 (postictal). Interictal: **0–5** y **26–30**.

Por qué solo de pacientes con crisis

Si el interictal saliera de cualquier paciente, el modelo podría aprender a **reconocer al paciente o al equipo** en vez del estado preictal. Con las dos clases de los mismos pacientes, ese atajo desaparece.

Ictal: todas las crisis

Las 2.662 crisis válidas, de inicio a fin, tengan o no lead-in (para saber el tipo no hace falta la señal de antes). Duración típica: mediana 35 s (p10 = 11 s, p90 = 123 s).



Fichero de ejemplo (🟩 absz (🔹), aaaaacrb_s001_t001): dos preictales, dos crisis y un interictal al final. El blanco es señal que no se usa.

Comprobaciones antes de guardar (sección 8)

El notebook se para con un `assert` si alguna falla. Las cinco dieron 0:

1. Todo tramo está dentro de su fichero.
2. Ningún preictal ni interictal se solapa con una crisis (ni con su SPH).
3. Ningún interictal cae a menos de 15 min antes de una crisis ni en su postictal.
4. Ningún paciente aparece en dos particiones (evita fuga de datos).
5. Ningún preictal empieza dentro de un margen postictal.

Qué hay en `tramos.csv` (una fila = un trozo de un EDF)

Columna	Qué es
`clase`	preictal, interictal o ictal
`split` , `patient` , `file`	partición, paciente y EDF
`montage` , `fs`	montaje y frecuencia originales (los necesita el 02)
`inicio_s` , `fin_s`	dónde empieza y acaba el tramo dentro del fichero
`tipo` , `grupo` , `crisis_id`	de qué crisis es (preictal e ictal)
`inicio_crisis_s`	inicio de la crisis: para recortar con el SPH de cada modelo
`cumple_Xmin` , `cumple_umbral_tipo`	si entra con cada umbral / con el de su tipo
`test_comun`	si pertenece al test común entre umbrales

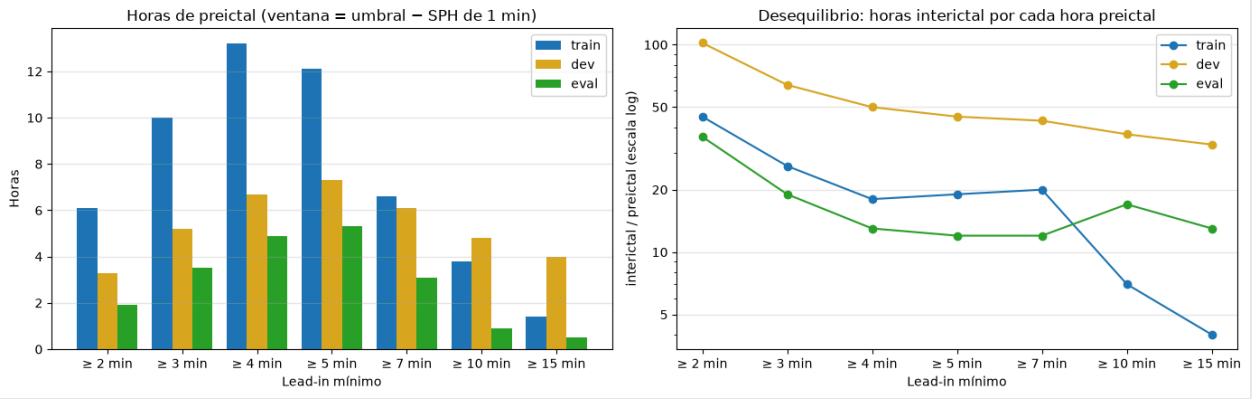
Tramos	🔵 train	🟡 dev	🟢 eval
ictal	1.810	492	360
interictal	2.274	1.485	549
preictal	374	206	115

El desequilibrio de clases (sección 9)

18 h
de interictal por cada hora de preictal
en 🔵 train (umbral 4 min)

50 h
en 🟡 dev

13 h
en 🟢 eval



Horas de preictal y desequilibrio por umbral.

Por qué importa

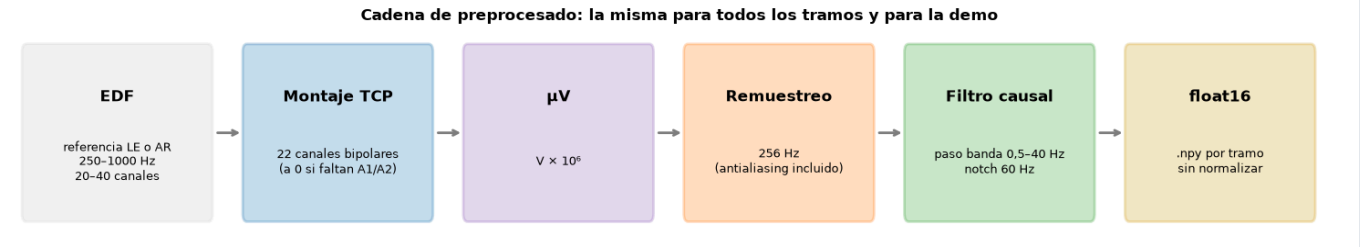
Un modelo que dijera siempre "normal" acertaría ~95 % y no avisaría nunca. Por eso se balancea **solo train** (en `04_particiones_y_balanceo`) y se evalúa con sensibilidad, falsas alarmas por hora y AUC-PR, **no con accuracy**.

3 · Notebook 02 · Preprocesado de la señal

Qué hace, en una frase

Lee **solo** los trozos de `tramos.csv` y los pasa por la **misma cadena**, para que el modelo vea siempre señal con la misma forma (22 canales × muestras a 256 Hz, en µV), venga del equipo que venga.

El problema: cada EDF de TUSZ es de un equipo distinto: referencia LE o AR, con o sin A1/A2, y a 250, 256, 400, 512 o 1000 Hz.



La cadena de preprocesado (`tfm_comun/senal/preprocesado.py`).

Paso	Qué hace	Por qué así
1. Montaje TCP `montaje.py`	Cada canal = resta de dos electrodos vecinos (`FP1-F7`, `F7-T3` …). Siempre 22 canales; si faltan A1/A2, esos 2 quedan a 0 y marcados como no válidos.	La resta elimina la referencia, que es lo que más cambia entre equipos. Forma fija = el modelo no se rompe.
2. A µV	× 10^6 (MNE da voltios).	En voltios, float16 perdería precisión.
3. Remuestreo `remuestreo.py`	A 256 Hz con `resample_poly` (p. ej. 250→256 = ×128/125). Incluye antialiasing.	Se hace **antes** de filtrar: así el filtro es siempre el mismo (diseñado a 256 Hz).
4. Filtro `filtrado.py`	Butterworth paso banda 0,5–40 Hz (orden 4) + notch 60 Hz (Q = 30). **Causal**.	Causal = solo usa el pasado, como en la demo en directo. El modelo se entrena con la misma señal que verá.
5. Guardado `senales.py`	Un `.npy` por tramo, float16, **sin normalizar**, recortado a ±60.000 µV.	Mitad de disco (~36 GB en vez de 72). Normalizar después permite probar varias opciones sin repetir esto.

El corazón: tres líneas

```
def preprocesar(senal_tcp_uv, fs_original, canales_validos=None):
    senal = filtrar(remuestrear(senal_tcp_uv, fs_original), FS_OBJETIVO).astype(np.float32)
    if canales_validos is not None:
        senal[~np.asarray(canales_validos)] = 0     # los que faltan, a 0
    return senal
```

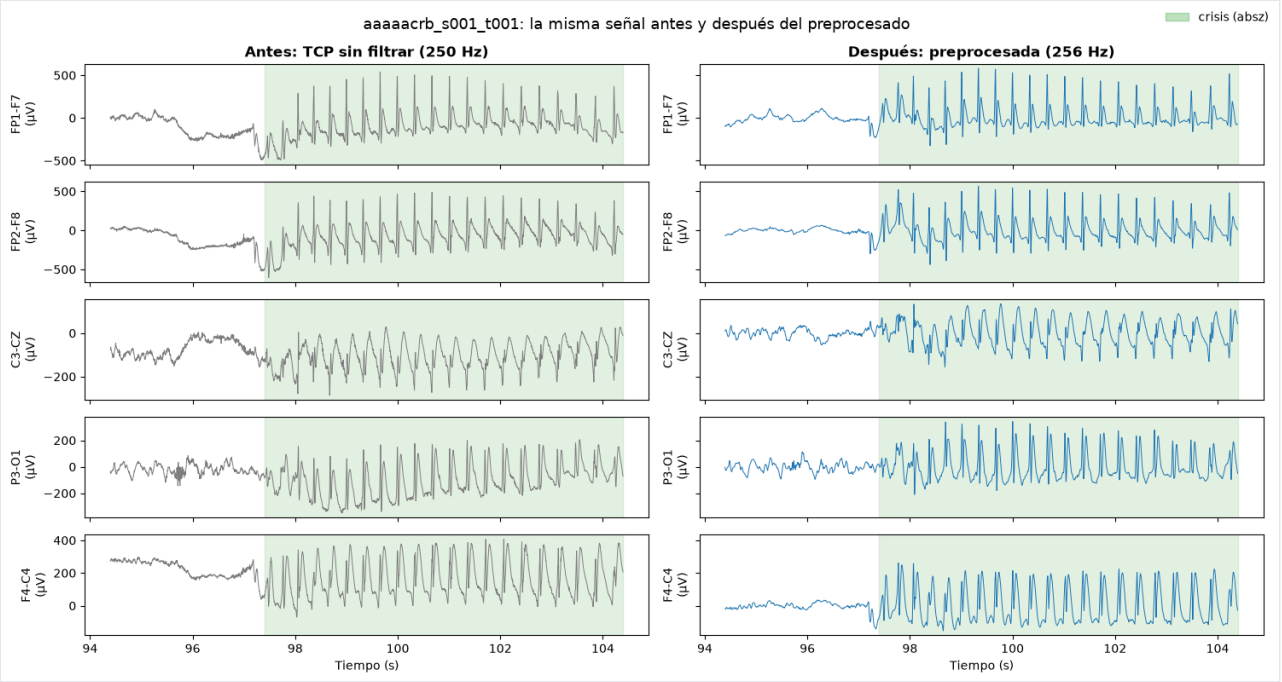
Pensado para la demo

Todo en `tfm_comun/senal/` son funciones de numpy/scipy **sin MNE** (salvo leer el EDF). `preprocesar_referencial()` sirve para la señal de un equipo real, y `FiltroContinuo` filtra a trozos guardando el estado: da exactamente lo mismo que filtrar de golpe.

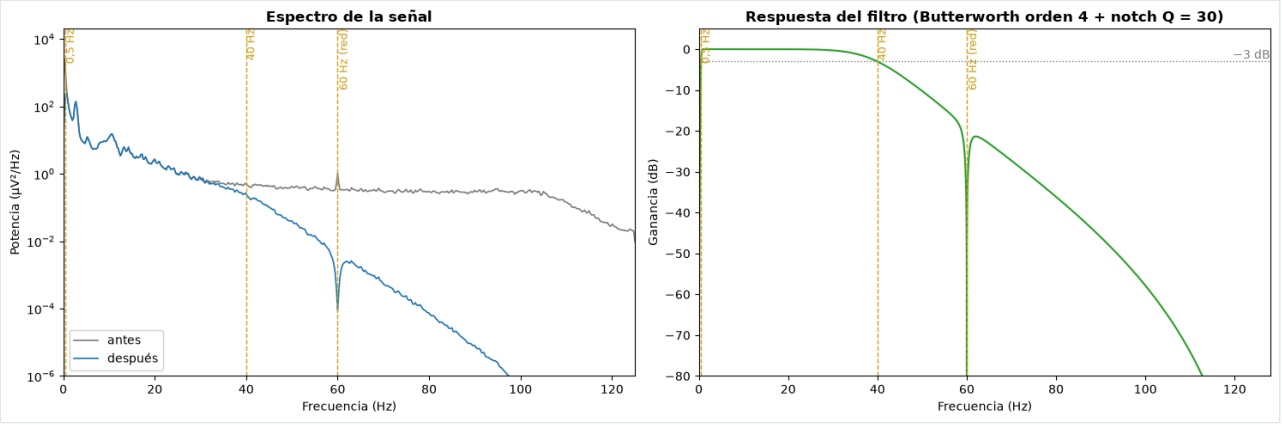
El recorrido, sección a sección

2 **Configuración.** `SOLO_PRUEBA` (procesar N ficheros para probar), `SOBRESCRIBIR`, `MARGEN_LECTURA_S = 10`. Carga `tramos.csv`, crea un `id_tramo` estable y localiza los 5.134 EDF (896,5 h de señal).

3. **¿Cabe en el disco?** Calcula lo que ocupará (22 × muestras × 2 bytes ≈ 36 GB, casi todo interictal) y se para si no hay un 10 % de margen libre.
4. **Prueba en un fichero de ejemplo** (la 🟩 absz (🔹) de la Fase 1): señal antes/después, espectro, respuesta del filtro y error de float16. Usa la **misma función** que el bucle.
5. **Bucle sobre todos los ficheros** (1–2 h).
6. `senales.csv` + 4 comprobaciones.
7. **Calidad de la señal** (solo se mide, no se descarta).
8. **Vista previa de la normalización** (se aplicará en el 03).



La misma ausencia antes (gris, 250 Hz) y después (azul, 256 Hz): se mantienen las puntas-onda y desaparece la deriva lenta.



Izquierda: el espectro cae fuera de 0,5–40 Hz y desaparece el pico de 60 Hz. Derecha: respuesta del filtro.

El corte es una cuesta, no un muro

A 40 Hz pasa el 71 % (−3 dB), a 50 Hz el 31 %, a 100 Hz el 0,1 %. Un corte brusco metería "ecos" junto a las puntas. A 60 Hz: −311 dB (eliminado).

float16 no pierde nada que importe

Error máximo en el ejemplo: **0,22 µV**, muy por debajo del ruido del EEG (unos µV). Hay un `assert` que exige < 1 µV.

El bucle (sección 5): rápido y reanudable

```
for fichero in tqdm(ficheros_pendientes):
    tramos_fichero = pendientes[pendientes.file == fichero]
    try:
        # 1. Lee UNA vez el EDF: del primer al último tramo, con 10 s de margen a cada lado
        raw = leer_edf_tusz(edf_de[fichero], tramos_fichero.inicio_s.min() - MARGEN_LECTURA_S,
                            tramos_fichero.fin_s.max() + MARGEN_LECTURA_S)
        # 2. Preprocesa todo lo leído
        senal, validos = preprocesar_raw(raw)
        # 3. Corta cada tramo, lo guarda en .npy y mide su calidad
        for t in tramos_fichero.itertuples():
            ...
        # 4. Lo apunta en progreso.csv (para reanudar)
    except Exception as e:
        errores.append(...)        # un fichero malo no para todo
```

Detalle	Por qué
Por **fichero**, no por tramo	Cada EDF se lee una sola vez aunque tenga varios tramos.
10 s de margen de lectura	El filtro necesita "arrancar": así el transitorio cae fuera del tramo.
`progreso.csv`	Si se corta (o se va la luz), al volver a ejecutar empieza donde lo dejó.
Guardado vía `.tmp` + renombrar	Nunca queda un `.npy` a medias.
`errores.csv`	Los ficheros que fallan se apuntan y se sigue.

Comprobaciones (sección 6)

1. Todos los tramos tienen su `.npy`.
2. Cada tramo tiene las muestras esperadas (duración × 256, ± 2).
3. 200 `.npy` al azar: forma 22 × muestras, float16, sin NaN ni infinitos.
4. Ficheros sin A1/A2 (`03_tcp_ar_a`): 20 canales válidos; el resto, 22.

`senales.csv` = `tramos.csv` + dónde está la señal + su calidad

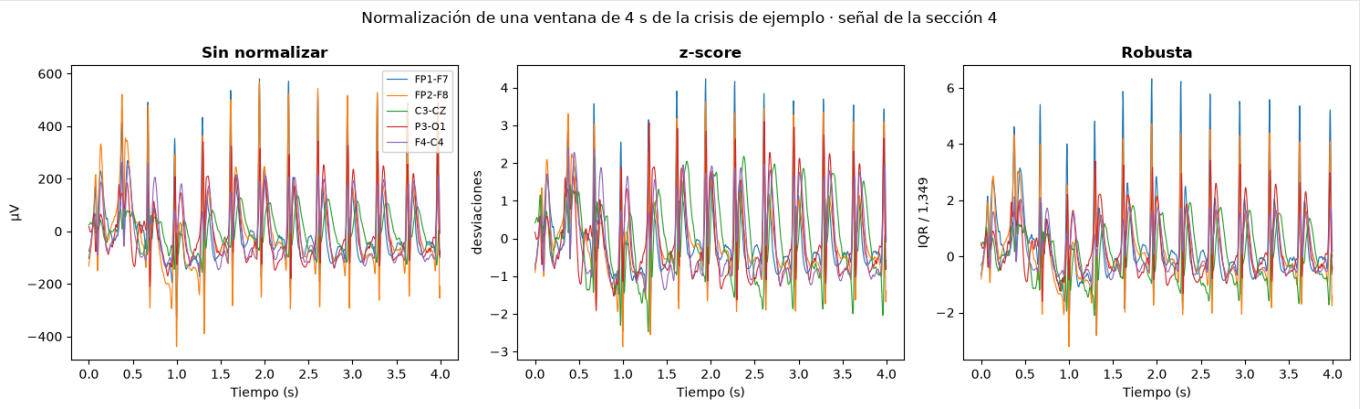
Columna nueva	Qué es
`id_tramo` , `ruta`	identificador y ruta del .npy: `<split>/<clase>/<id_tramo>.npy`
`n_muestras` , `fs_original`	muestras a 256 Hz y frecuencia original
`mascara_canales`	un 1/0 por canal TCP (0 = no se pudo construir)
`amplitud_max_uv`	pico del tramo: muy alto = posible artefacto
`desviacion_mediana_uv`	nivel típico de la señal
`canales_planos`	canales con desviación < 0,5 µV (electrodo suelto)
`canales_validos` , `fraccion_recortada`	22 o 20; fracción de muestras recortadas a ±60.000 µV

Calidad (sección 7): se mide, no se descarta

El umbral de rechazo se decidirá en `03_ventanas` y tendrá que ser **alto (≥ 1.000–1.500 µV)**: las 🟩 absz (🔹) tienen puntas-onda grandes y con 300–600 µV se perderían.

Normalización (sección 8): vista previa

Se normalizará **cada ventana y cada canal por separado** en el 03, para que el modelo no dependa de la amplitud de cada paciente o equipo. En la demo bastará con la ventana que acaba de llegar.



Una ventana de 4 s de la crisis de ejemplo: sin normalizar, z-score y robusta.

z-score

(x − media) / desviación típica.

Robusta

(x − mediana) / (IQR / 1,349). Un artefacto enorme no aplasta el resto de la ventana.

Las dos se compararán en la Fase 3.

🔴 Estado de la ejecución guardada del 02

El notebook está guardado con `SOLO_PRUEBA = 20` y `SOBRESCRIBIR = True`: procesó 20 ficheros (103 tramos, 0 errores), no los 7.665. Las tablas de calidad que muestra son de esa prueba. Para la ejecución completa: `SOLO_PRUEBA = None` y `SOBRESCRIBIR = False` (así además reanuda si se corta).

Archivos generados (`outputs/fase_02/02_preprocesado/`)

- `senales/<split>/<clase>/*.npy`: la señal. ⚠️ Decenas de GB: no subir a GitHub.
- `senales.csv`: entrada de `03_ventanas`.
- `configuracion.json`, `progreso.csv`, `errores.csv`, `summary.txt`, `figuras/`.

4 · Resumen para la defensa

Decisión	Por qué
Umbral 4 min = 3 preictal + 1 SPH	Máxima señal preictal en train con ≥ 20 pacientes en dev y eval.
Margen postictal de 5 min (0 en ausencias)	El preictal no empieza con EEG alterado por la crisis anterior.
Ausencias con valores propios (60 s / 10 s / 0 s)	Son cortas y seguidas: con 5 min casi no quedarían.
Interictal a > 15 min de cualquier crisis	Es el mismo para todos los umbrales; nunca puede ser preictal.
Interictal solo de pacientes con crisis	Evita que el modelo aprenda a reconocer pacientes.
Tramo preictal más largo posible	Una tabla sirve para todos los modelos y umbrales.
Test común entre umbrales	Comparación justa: mismas crisis, mismo minuto.
Montaje TCP de 22 canales fijo	Elimina la referencia; misma forma siempre.
Remuestrear antes de filtrar	Un único filtro, diseñado a 256 Hz.
Filtro causal	Reproducible en directo: misma señal en entrenamiento y demo.
float16 sin normalizar	Mitad de disco, error despreciable; normalización intercambiable.